`10`; if the current value is `8`, sending an `Increment(1)` responds with `9`.

To call an actor from a stream, `ask` via the `ActorFlow` class, which expects an actor referenced and a message:

```
import akka.stream.typed.scaladsl.ActorFlow

class ActorSpec ... {

  ...

  ActorFlow.ask(ref)((elem: Int, replyTo: ActorRef[Int]) =>
    Cap.Increment(elem, replyTo))
  ...
}
```

The first part, `ask(ref)`, is simple: pass a reference to the actor you want to ask. The second part is not so simple. The signature is a function that converts a tuple into an actor's message: here, `(Int, ActorRef[Int]) => Increment`. The tuple is formed from an element of the stream—here, integers—and an `ActorRef`. This `ActorRef` is not the actor you are asking, but a proxy between the stream and that actor. It's a proxy that Akka creates for you. You may recall that this is also the case with a simple `ask` from one actor to another. Following is a test that creates a `Cap` actor and a flow with a source of integers that call this actor.

Listing 16.4 Connecting to an actor

```
class ActorSpec ... {
  ...

  "connecting to an actor from a stream" should {
    "send each element, wait and get back the answer from the actor" in {

    val ref = spawn(Cap(0, 3))                    ❶
```